AGENTIC AI

BUILD & DEPLOY

COURSE SLIDES · WSQ

Build and Deploy Agentic AI Apps with CrewAI, Autogen, ADK and Streamlit

WSQ Course Code: TGS-2025059028 · 4-Day Hands-On Training

Skills Framework TSC: Generative AI Application Development and Deployment (ICT-INT-0047-1.1)

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v2 · 11 August 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

Digital Attendance (Mandatory)

1. Trainer displays the SSG digital attendance QR code
2. Scan it with your mobile phone camera
3. Submit your attendance — AM, PM and Assessment
4. Repeat every session, every day
5. 75% attendance minimum for assessment and funding

Digital attendance is mandatory for WSQ-funded courses — missing a scan puts your funding at risk.

© 2026 Tertiary Infotech Academy Pte Ltd

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE

Principal Trainer, Tertiary Infotech Academy Pte Ltd

BACKGROUND

PhD — 20+ years of industry and training experience in software engineering, data science and applied AI.

DELIVERS

WSQ courses on Generative AI, Agentic AI, RAG, machine learning and full-stack application development.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

1

Who you are

Your name, your organisation and your role.

2

What you've built

Your experience with Python, LLM APIs or chatbots — beginners welcome.

3

What you want

One task at your workplace you would like an AI agent to take over.

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

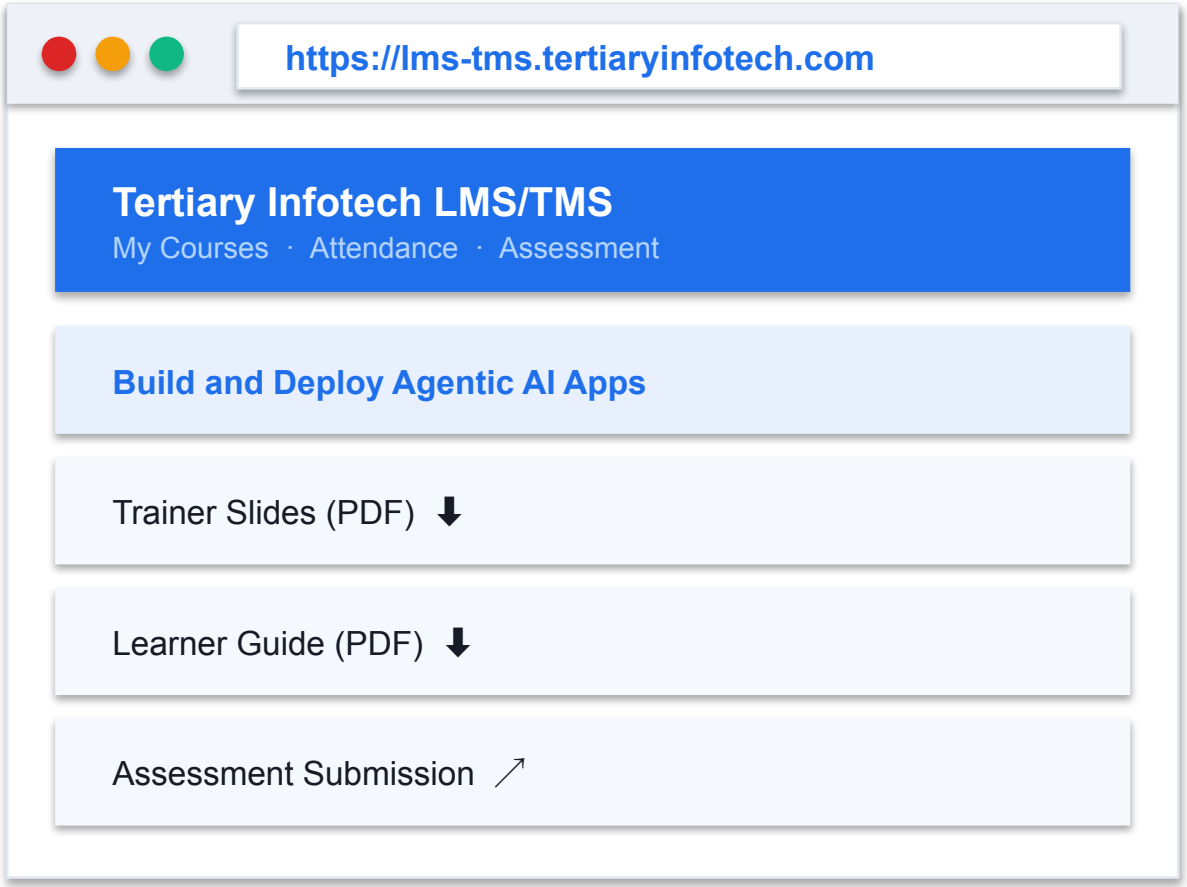
Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. 75% attendance is required.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE PORTAL · LMS/TMS

Download Course Material



1 Sign in
lms-tms.tertiaryinfotech.com — log in with your registered email (OTP or password).

2 Open your course
Select 'Build and Deploy Agentic AI Apps with CrewAI, Autogen, ADK and Streamlit' under My Courses.

3 Download materials
Slides and the Learner Guide — your open-book references for the assessment.

4 Submit & survey
Upload your assessment answers and complete the TRAQOM survey QR here.

All course material is downloaded from the LMS/TMS portal — keep it handy: the final assessment is open book.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL USE · ALL 20 ACTIVITIES

Your Development Toolkit

1

Python 3.11+ and a virtual environment

Every activity runs in an isolated .venv so dependencies never collide.

2

The three agent frameworks

CrewAI, Microsoft AutoGen and the OpenAI Agents SDK / ADK — installed on Day 1.

3

Streamlit

Turns a Python script into a shareable web app, then into a public cloud deployment.

4

ChromaDB + embeddings

The vector store behind the RAG applications you build in LU2.

5

API keys in .env

OpenAI, Tavily and OpenRouter keys — kept out of source control from the very first activity.

6

An AI IDE

GitHub Copilot, Cursor, Trae or Windsurf to accelerate the code you write.

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE

Lesson Plan — 4 Days, 9:30am–6:30pm

Days 1–2 — build agents

- **Day 1 — Agentic AI Foundations & Your First Agent**
  - LU1: frameworks, agent anatomy · Activities 1–3
- **Day 2 — Multi-Agent Workflows & AI-Assisted Development**
  - LU1: CrewAI, AutoGen, orchestration, AI IDEs · Activities 4–7
- **Daily timing**
  - 9:30am–6:30pm · 1-hour lunch · tea breaks within

Days 3–4 — deploy & safeguard

- **Day 3 — RAG, Context Augmentation & Cloud Deployment**
  - LU2: RAG, knowledge graphs, cloud deployment · Activities 8–14
- **Day 4 — Guardrails, Model Alignment & Assessment**
  - LU3: guardrails, alignment, production · Activities 15–20
- **Final Assessment (Day 4)**
  - WA (SAQ) 1 hour + PP (practical) 1 hour — open book

32 training hours across 4 days (8 hours per day) plus a 2-hour final assessment on Day 4.

© 2026 Tertiary Infotech Academy Pte Ltd

WSQ ALIGNMENT · ICT-INT-0047-1.1

Skills Framework Alignment

TSC element	Detail
TSC Title	Generative AI Application Development and Deployment
TSC Code	ICT-INT-0047-1.1
Proficiency	Level 1
Ability A5	Design integration workflows between AI models and systems, such as batch processing, API integration, and multi-model orchestration
Ability A2	Resolve compatibility issues with other software or systems (e.g. GitHub Copilot, CircleCI)
Ability A3	Identify issues in generative AI models by analysing and interpreting error logs
Knowledge K1	Concepts of software integration
Knowledge K2	Concepts of integrating multiple generative AI models, including common compatibility and integration issues
Knowledge K3	Concepts behind context augmentation techniques, such as knowledge graphs

Every topic, activity and assessment item maps to these TSC ability and knowledge statements.

© 2026 Tertiary Infotech Academy Pte Ltd

BY THE END OF THIS COURSE

Learning Outcomes

1

LO1

Design and implement multi-agent integration workflows using Agentic AI frameworks — CrewAI, AutoGen and the OpenAI Agents SDK / ADK

2

LO2

Develop and deploy Agentic AI and RAG applications for inference on Streamlit Cloud

3

LO3

Deploy guardrails and model alignment techniques to ensure safe and aligned outputs in Agentic AI applications

© 2026 Tertiary Infotech Academy Pte Ltd

THE JOURNEY

Course Outline — Three Learning Units

Days 1–2

- **LU1 — Agentic AI App Development**
  - Frameworks: CrewAI · AutoGen · OpenAI Agents SDK / ADK
  - Multi-agent workflows and workflow design patterns
  - Building agentic apps · development with AI IDEs
  - Activities 1–7
- **LU2 — Agentic AI and RAG Deployment**
  - Context augmentation · knowledge graphs · GraphRAG

Days 3–4

- Cloud inference · error-log analysis · Streamlit Cloud
- Activities 8–14
- **LU3 — Model Alignment and Guardrails**
  - Safeguarding LLMs with guardrails
  - Model alignment for safety · RLHF, DPO, Constitutional AI
  - Deploying guardrails on Agentic AI applications
  - Activities 15–20

© 2026 Tertiary Infotech Academy Pte Ltd

CONCRETE DELIVERABLES

20 Hands-On Activities — What You'll Ship

1. **A tool-using agent**
   A Streamlit chatbot that searches the live web (Activity 3).

2. **A CrewAI crew and an AutoGen team**
   Role-based and conversational multi-agent systems (Activities 4–5).

3. **A multi-agent orchestrator**
   Trip planner delegating to three specialists (Activity 6).

4. **A RAG application**
   Invoice question-answering grounded in your own PDFs (Activity 8).

5. **A public cloud deployment**
   Your app live on a *.streamlit.app URL (Activities 12, 14).

6. **A guardrailed production app**
   Input, output and human-oversight controls that survive a red-team re-test (Activity 20).

© 2026 Tertiary Infotech Academy Pte Ltd

BEFORE THE ASSESSMENT

Briefing for Assessment

1

Phones away

Place phones and other materials under the table or on the floor.

2

No recording

No photos or recording of the assessment scripts.

3

Work alone

No discussion with other learners during the assessment.

4

Black or blue pen

Use a black or blue pen for hard-copy assessments.

5

No correction fluid

No liquid paper or correction tape — cross out cleanly instead.

6

Time is called

Assessment scripts are collected when the time is up.

© 2026 Tertiary Infotech Academy Pte Ltd

Final Assessment

WA — Written

- Short-Answer Questions (SAQ)
- 1 hour · open book
- Tests the KNOWLEDGE behind the course
- Covers all three Learning Units

PP — Practical

- Practical Performance tasks
- 1 hour · open book
- Build, deploy or safeguard an app
- Drawn from the class activities

Rules & outcome

- Open book: slides, Learner Guide and approved materials
- Grading: Competent / Not Yet Competent
- 75% attendance required to be eligible
- An appeal process is available

© 2026 Tertiary Infotech Academy Pte Ltd

ON ASSESSMENT DAY

Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Sit WA (SAQ) 1 h, then PP 1 h — open book
4. Submit your answers on the LMS
5. Sign the Assessment Summary Record

© 2026 Tertiary Infotech Academy Pte Ltd

01

LEARNING UNIT 01

Agentic AI App Development with Agentic AI Frameworks

Agentic AI frameworks · multi-agent workflows · workflow design · building agentic apps · development with AI IDEs

© 2026 Tertiary Infotech Academy Pte Ltd

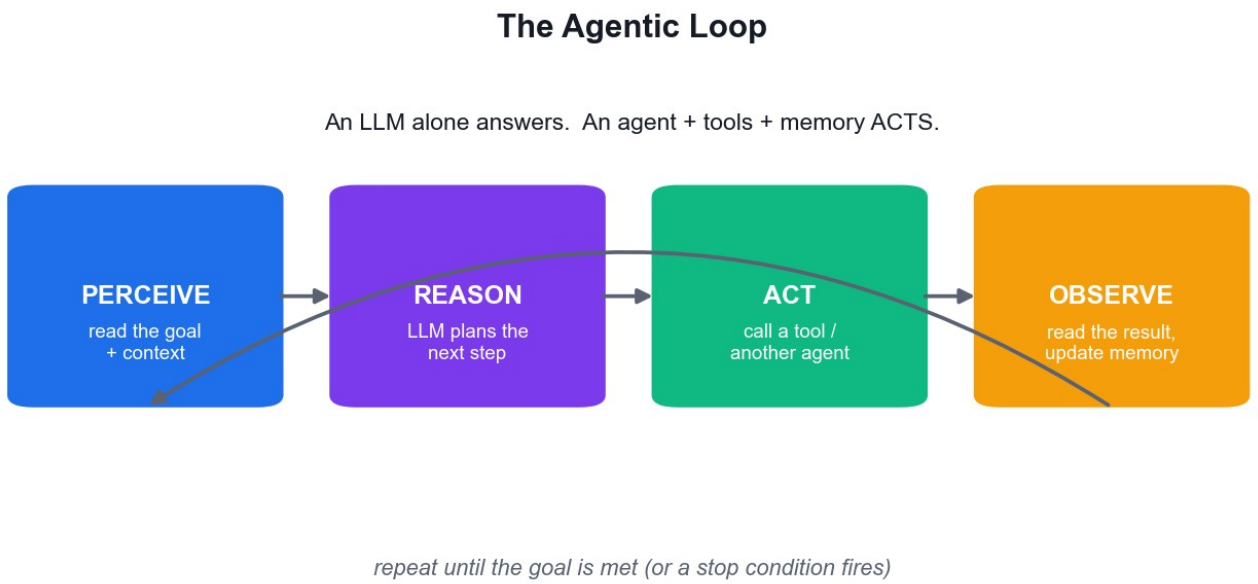
WHY AGENTIC AI

An LLM answers. An agent acts.

Give a model instructions, tools and memory, and it stops being a chatbot and starts completing goals on your behalf.

© 2026 Tertiary Infotech Academy Pte Ltd

The Agentic Loop



Perceive
The agent reads its goal, the conversation so far and any retrieved context.

Reason
The LLM decides the next step — answer directly, or call a tool first.

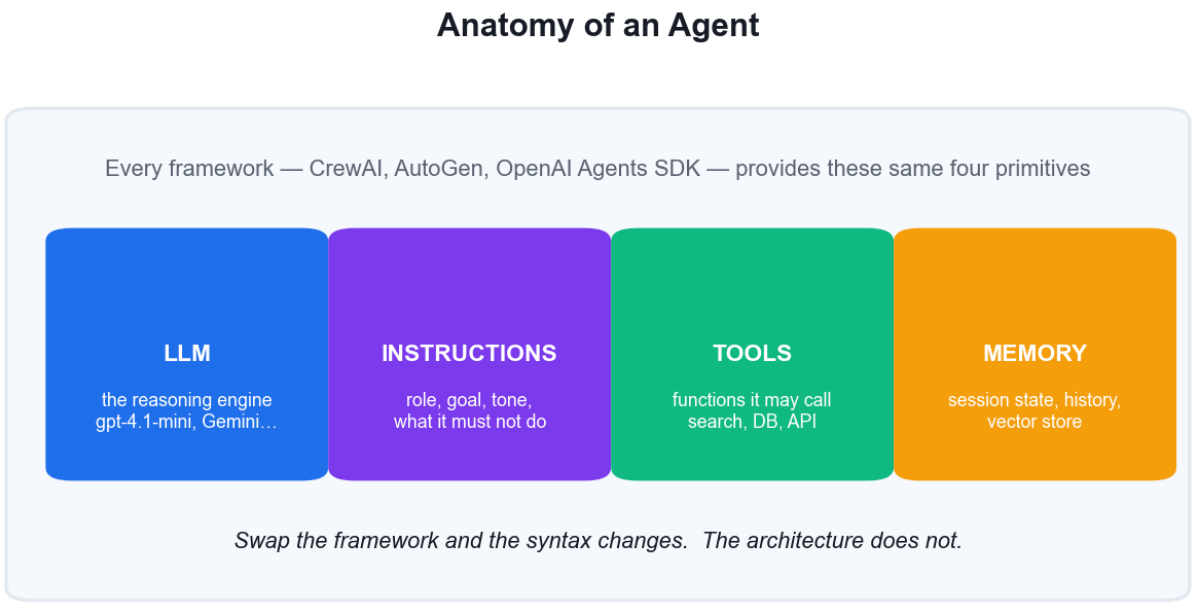
Act
It invokes a function tool, another agent, or returns the final answer.

Observe
The tool result re-enters the context and the loop repeats until done.

© 2026 Tertiary Infotech Academy Pte Ltd

FOUR PRIMITIVES · EVERY FRAMEWORK

Anatomy of an Agent



LLM
The reasoning engine — gpt-4.1-mini, Gemini, or any OpenAI-compatible model.

Instructions
Role, goal, tone and hard limits. This is your primary control surface.

Tools
The functions it may call. The docstring is what the model reads to choose.

Memory
Session state and history so the agent knows what already happened.

© 2026 Tertiary Infotech Academy Pte Ltd

Agentic AI Frameworks — the Landscape

1

CrewAI

Role-based crews. Declarative agents with a role, goal and backstory collaborating on tasks.

2

Microsoft AutoGen

Conversational multi-agent teams with first-class human-in-the-loop support.

3

OpenAI Agents SDK / ADK

Lightweight and production-oriented: Agent, Runner, function tools, handoffs, guardrails.

4

LangGraph

Graph-based state machines for workflows needing explicit, inspectable control flow.

5

Google ADK

Agent Development Kit for building and deploying agents on Google Cloud / Vertex AI.

6

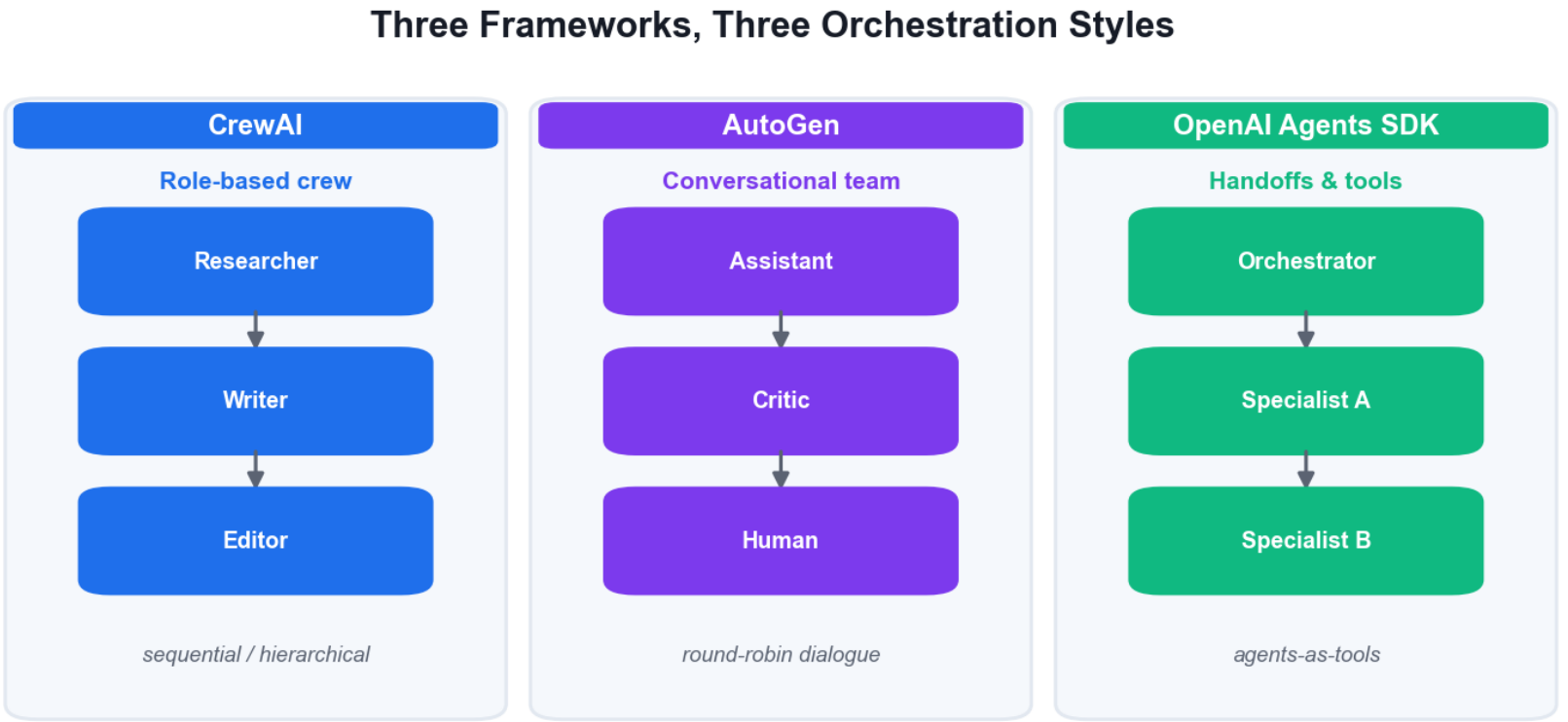
Common ground

All of them wrap the same loop — pick for orchestration style and ecosystem, not hype.

© 2026 Tertiary Infotech Academy Pte Ltd

HOW THEY DIFFER

Three Frameworks, Three Orchestration Styles



CrewAI models a team by ROLE. AutoGen models work as a CONVERSATION. The OpenAI Agents SDK models it as HANDOFFS and agents-as-tools.

© 2026 Tertiary Infotech Academy Pte Ltd

LU1 · T1 — DECISION TABLE

Choosing a Framework

Criterion	CrewAI	AutoGen	OpenAI Agents SDK
Mental model	Role-based crew	Agent conversation	Handoffs + tools
Best for	Process-shaped work	Debate, review, critique	Production apps & APIs
Human-in-the-loop	Limited	Built in	Via guardrails
Learning curve	Low	Medium	Low
Multi-model	Yes	Yes	Yes, via base_url
Structured output	Pydantic	Pydantic	Pydantic output_type

There is no universal winner. Score the frameworks against YOUR requirements — that is Activity 2.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 1 · 45 MIN

Activity 1 — Set Up the Agentic AI Development Environment

Prepare the Python environment every later activity depends on: a virtual environment, the agent frameworks, and the API keys for OpenAI, Tavily and OpenRouter stored safely in a .env file that is never committed to source control.

1. Create and activate .venv
2. pip install the frameworks
3. Get OpenAI + Tavily API keys
4. Save keys into .env
5. Verify with a version check

YOU'LL PRODUCE A working .venv with CrewAI, AutoGen, the OpenAI Agents SDK and Streamlit installed, plus a .env holding your API keys.

Tools: Python 3.11+, pip, venv, VS Code, OpenAI Platform, Tavily, OpenRouter · Full step-by-step guide: Learner Guide, Activity 1

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON ACTIVITY

ACTIVITY 2 · 30 MIN

Activity 2 — Compare CrewAI, AutoGen and the OpenAI Agents SDK

Before writing agent code, decide which framework fits a given problem. Score all three frameworks against a real requirement set — orchestration style, human-in-the-loop, tool ecosystem, learning curve and production readiness — and justify a recommendation.

1. Write your use-case requirements
2. Score each framework 1-5
3. Weight the criteria that matter
4. Total the scores
5. Justify your recommendation

YOU'LL PRODUCE A completed framework-comparison matrix with a written, evidence-based recommendation for one named use case.

Tools: Framework documentation, comparison matrix template (labs/data/framework_matrix.csv) · Full step-by-step guide: Learner Guide, Activity 2

© 2026 Tertiary Infotech Academy Pte Ltd

Tea Break

15 minutes

From One Agent to Many

1

A single agent has limits

One prompt carrying every instruction becomes long, contradictory and hard to debug.

2

Specialisation wins

A narrow agent with 2–3 relevant tools outperforms one agent with fifteen.

3

Separation of concerns

Each agent can be tested, prompted and improved independently.

4

An orchestrator decides

A coordinating agent routes work to the right specialist at run time.

5

Context is the currency

Multi-agent design is really about controlling what each agent sees.

6

The cost is coordination

More agents means more calls, more latency and more ways to fail — justify each one.

© 2026 Tertiary Infotech Academy Pte Ltd

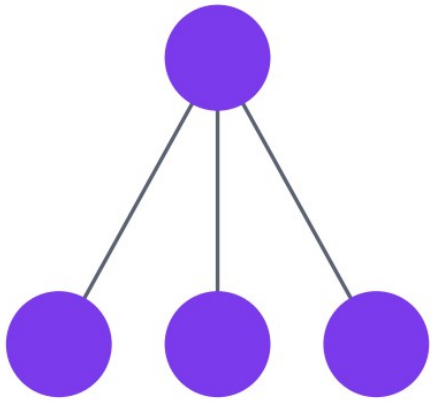
Four Multi-Agent Orchestration Patterns

Sequential



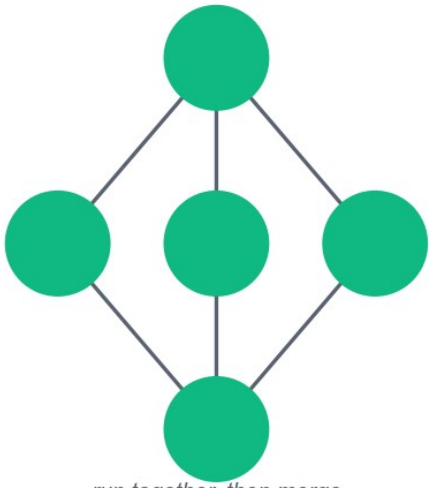
each agent feeds the next

Router–Worker



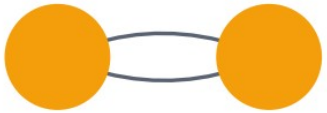
router picks the specialist

Parallel fan-out



run together, then merge

Reflection



generate, critique, repeat

Sequential for pipelines · Router–worker for triage · Parallel fan-out for independent subtasks · Reflection for quality. Most real systems combine two or more.

© 2026 Tertiary Infotech Academy Pte Ltd

Single-Agent vs Multi-Agent

Simpler is better

- **Use a SINGLE agent when**
  - The task is one coherent job
  - A handful of tools is enough
  - Latency and cost matter most
  - You want the simplest thing that works

Complexity that earns its keep

- **Use MULTI-AGENT when**
  - Distinct expertise is genuinely needed
  - Subtasks can run in parallel
  - You need a critic or reviewer role
  - One prompt has become unmaintainable

Start single-agent. Split into multiple agents only when you can name the specific problem it solves.

© 2026 Tertiary Infotech Academy Pte Ltd

Designing an Agentic Workflow

1

1 · Define the goal

State the outcome precisely, including how you will know it succeeded.

2

2 · Decompose

Break the goal into subtasks a specialist could own end to end.

3

3 · Assign tools

Give each agent only the tools its subtask needs — nothing more.

4

4 · Choose the pattern

Sequential, router, parallel or reflection — match it to the dependency shape.

5

5 · Define handoffs

Specify exactly what data passes between agents, and in what structure.

6

6 · Add checks

Structured output, validation and a stop condition so it cannot loop forever.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 3 · 60 MIN

Activity 3 — Build a Single Agent with Tools (OpenAI Agents SDK)

Build the course's first working agent: a Streamlit chatbot backed by the OpenAI Agents SDK that can search the live internet with a Tavily function tool. This is labs/app/simple_chat.py and it introduces Agent, Runner, @function_tool and Streamlit session state.

1. Define the @function_tool
2. Create the Agent with instructions
3. Wire Streamlit chat + session state
4. Run with Runner.run
5. Ask a current-events question

YOU'LL PRODUCE A running Streamlit chat app whose agent calls a Tavily search tool to answer questions about current events.

Tools: OpenAI Agents SDK, Tavily API, Streamlit, labs/app/simple_chat.py · Full step-by-step guide: Learner Guide, Activity 3

© 2026 Tertiary Infotech Academy Pte Ltd

Lunch Break

1 hour

ACTIVITY 4 · 60 MIN

Activity 4 — Build a Role-Based Crew with CrewAI

Model a content-production team as a CrewAI crew: a Researcher agent gathers facts and a Writer agent turns them into an article. Each agent gets a role, goal and backstory, and the tasks run in a sequential process where task 2 consumes task 1's output.

1. Define Researcher + Writer agents
2. Write the two Tasks
3. Assemble the Crew (sequential)
4. crew.kickoff() with inputs
5. Read the handed-off output

YOU'LL PRODUCE A Python script that runs a two-agent CrewAI crew and prints a finished article on a topic you choose.

Tools: CrewAI, OpenAI API, labs/app/crew_content.py · Full step-by-step guide: Learner Guide, Activity 4

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 5 · 60 MIN

Activity 5 — Build a Conversational Team with AutoGen

Use Microsoft AutoGen to solve a problem through a structured conversation between two agents. A primary assistant proposes a solution and a critic reviews it, looping until the critic approves — the reflection pattern implemented as agent dialogue.

1. Create the model client
2. Define assistant + critic agents
3. Set the termination condition
4. RoundRobinGroupChat .run_stream
5. Watch the critique loop

YOU'LL PRODUCE An AutoGen round-robin team where an assistant and a critic iterate on a task until a termination condition fires.

Tools: AutoGen AgentChat, OpenAI API, labs/app/autogen_team.py · Full step-by-step guide: Learner Guide, Activity 5

© 2026 Tertiary Infotech Academy Pte Ltd

Building Production Agentic Apps

1

Structured output

Pydantic models turn free text into data your UI and code can rely on.

2

Tool docstrings are prompts

The description is what the model reads to decide when to call — write it carefully.

3

Handle failure explicitly

Tools time out and APIs rate-limit. Catch, retry with backoff, and degrade gracefully.

4

Bound the loop

Always set a maximum number of turns so an agent cannot run indefinitely.

5

Instrument everything

Log every tool call and decision — you cannot debug what you did not record.

6

Control the spend

Cache repeated calls and set a hard provider spend limit before you go public.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 6 · 75 MIN

Activity 6 — Orchestrate a Multi-Agent Trip Planner (Agents-as-Tools)

Build the course's flagship multi-agent app: a Streamlit trip planner where an orchestrator delegates to Planner, Budget and Local Guide agents exposed via the .as_tool() pattern, each with its own Tavily search tool and Pydantic-structured output. This is labs/app/trip_planner.py.

1. Review the 3 specialist agents
2. See .as_tool() delegation
3. Note the Pydantic output models
4. Run the Streamlit app
5. Plan a real trip and read the tabs

YOU'LL PRODUCE A Streamlit app that produces a day-by-day itinerary, a budget breakdown and local food tips from three specialist agents.

Tools: OpenAI Agents SDK, Tavily API, Pydantic, Streamlit, labs/app/trip_planner.py · Full step-by-step guide: Learner Guide, Activity 6

© 2026 Tertiary Infotech Academy Pte Ltd

Tea Break

15 minutes

Development with AI IDEs

1

GitHub Copilot

Inline completion, chat and multi-file edits inside VS Code and JetBrains.

2

Cursor

An AI-first fork of VS Code with whole-codebase context and agentic multi-file edits.

3

Windsurf

Agentic 'flows' that plan and execute changes across files, with fast iteration.

4

Trae

Autonomous, task-level development — describe the outcome and review the produced diff.

5

Where they excel

Boilerplate, tool definitions, tests, refactors and explaining unfamiliar code.

6

Where you must verify

Hallucinated APIs, outdated SDK syntax, silent logic errors and leaked secrets.

© 2026 Tertiary Infotech Academy Pte Ltd

THE WORKING RULE

AI-assisted, human-verified.

An AI IDE drafts faster than you can type — but you remain accountable for every line it produces. Read the diff.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 7 · 45 MIN

Activity 7 — Accelerate Agent Development with an AI IDE

Use an AI IDE — GitHub Copilot, Cursor, Trae or Windsurf — to extend an existing agent. Generate a new function tool from a comment, use the chat panel to explain unfamiliar agent code, and let the IDE diagnose a deliberately broken tool definition.

1. Install and sign in to the AI IDE
2. Ask chat to explain the agent code
3. Comment-driven tool generation
4. Review and correct the draft
5. Run and confirm the tool fires

YOU'LL PRODUCE A new, working function tool added to simple_chat.py that was drafted by the AI IDE and verified by you.

Tools: GitHub Copilot / Cursor / Trae / Windsurf, VS Code, labs/app/simple_chat.py · Full step-by-step guide: Learner Guide, Activity 7

© 2026 Tertiary Infotech Academy Pte Ltd

LU1 · LEARNING OUTCOME 1 ACHIEVED

Recap — Learning Unit 1

- You can explain the agentic loop and the four primitives every framework provides.
- You can compare CrewAI, AutoGen and the OpenAI Agents SDK and justify a choice.
- You have built a tool-using agent, a CrewAI crew and an AutoGen conversational team.
- You have orchestrated specialists with the agents-as-tools pattern.
- You can use an AI IDE to accelerate agent development — and verify what it writes.

© 2026 Tertiary Infotech Academy Pte Ltd

02

LEARNING UNIT 02

Agentic AI and RAG Deployment on Streamlit Cloud

Context augmentation & knowledge graphs · cloud inference · error-log analysis · deploying RAG and Agentic AI apps

© 2026 Tertiary Infotech Academy Pte Ltd

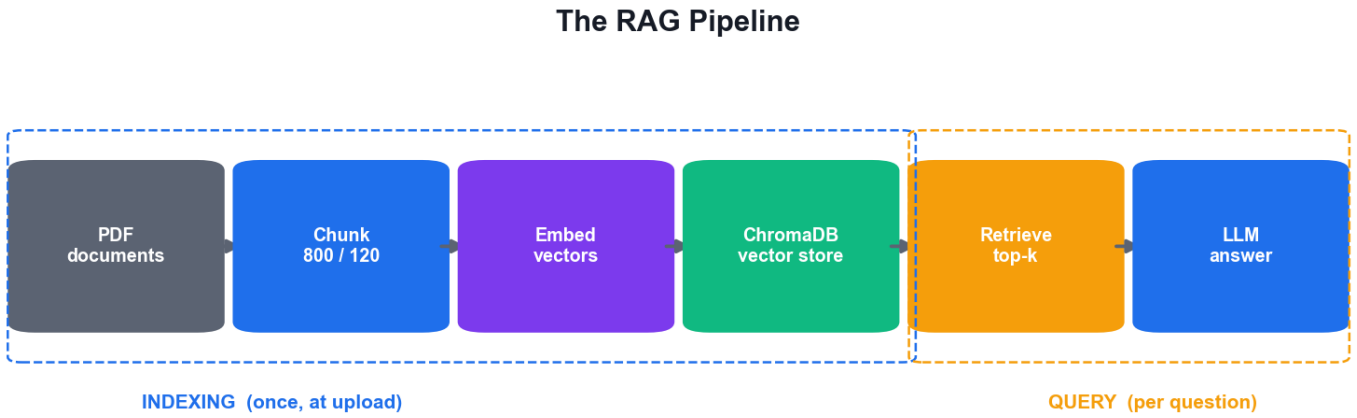
WHY RAG

The model does not know your data.

RAG closes the gap: retrieve the right context at query time and the answer becomes grounded, current and citable.

© 2026 Tertiary Infotech Academy Pte Ltd

The RAG Pipeline



The model never sees your whole corpus — only the top-k chunks most similar to the question.

Chunk with overlap

800 characters with 120 overlap keeps sentences whole across boundaries.

Embed into vectors

Text becomes a vector; similar meaning lands close together in the space.

Store and retrieve

ChromaDB returns the top-k nearest chunks to the question vector.

Augment, then generate

The retrieved chunks are injected into the prompt — the model answers from them.

© 2026 Tertiary Infotech Academy Pte Ltd

Context Augmentation Techniques

1

Vector RAG

Semantic similarity search over embedded chunks — the default, and the easiest to build.

2

Knowledge graphs

Entities and relationships enable multi-hop reasoning that similarity alone cannot do.

3

GraphRAG

Combines both: retrieve by similarity, then traverse relationships to expand the context.

4

Hybrid search

Vector similarity plus keyword/BM25 — catches exact identifiers that embeddings miss.

5

Re-ranking

A second, more precise model reorders candidates so the best chunk reaches the prompt.

6

Metadata filtering

Restrict retrieval by date, source or customer before similarity is even computed.

© 2026 Tertiary Infotech Academy Pte Ltd

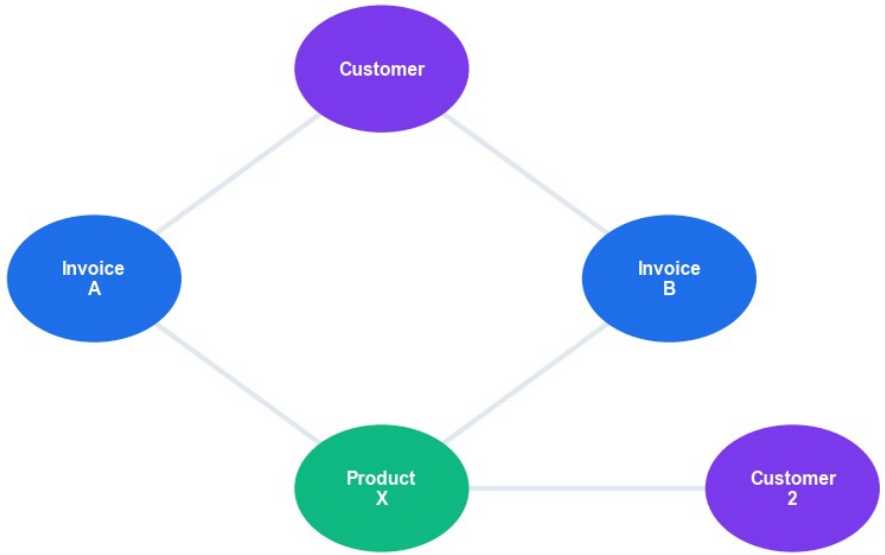
Vector RAG vs GraphRAG

Vector RAG — flat similarity



Chunks are independent.
No link between them — single-hop only.

GraphRAG — entities + relationships



Traversable links answer multi-hop questions
flat similarity cannot.

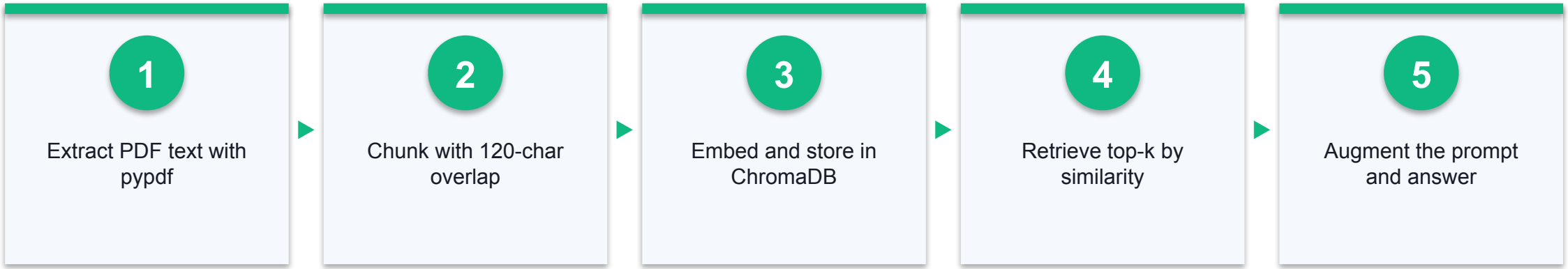
Flat chunks answer 'what is on invoice 47905?'. Only a graph reliably answers 'which OTHER customers bought the same product?' — the multi-hop question.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 8 · 75 MIN

Activity 8 — Build a RAG Pipeline over Invoice PDFs

Build the full Retrieval-Augmented Generation pipeline end to end: extract text from invoice PDFs, chunk it with overlap, embed the chunks, store them in ChromaDB, and answer natural-language questions grounded in the retrieved context. This is labs/app/invoice_rag.py.



YOU'LL PRODUCE A Streamlit app that indexes the sample invoice PDFs and answers questions about them with grounded, cited answers.

Tools: OpenAI Embeddings, ChromaDB, pypdf, Streamlit, labs/app/invoice_rag.py, labs/app/fakes invoices/ · Full step-by-step guide: Learner Guide, Activity 8

© 2026 Tertiary Infotech Academy Pte Ltd

Tea Break

15 minutes

ACTIVITY 9 · 45 MIN

Activity 9 — Tune Retrieval Quality — Chunking and Top-K

Retrieval quality decides answer quality. Run a controlled experiment across chunk sizes and top-k values on the same question set, record which configuration retrieves the right context, and choose settings you can defend with evidence.

1. Write 5 gold-standard questions
2. Test chunk 300 / 800 / 1500
3. Test top-k 1 / 3 / 5
4. Record hits and misses
5. Pick and justify the settings

YOU'LL PRODUCE A completed retrieval-tuning table showing accuracy at three chunk sizes and three top-k values, with a chosen configuration.

Tools: ChromaDB, OpenAI Embeddings, labs/data/retrieval_tuning.csv · Full step-by-step guide: Learner Guide, Activity 9

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 10 · 45 MIN

Activity 10 — Context Augmentation with a Knowledge Graph

Vector similarity alone cannot answer multi-hop questions that require joining facts. Model the invoice data as entities and relationships, then answer a question that plain RAG gets wrong — the core argument for GraphRAG and knowledge-graph context augmentation.

1. Extract entities from invoices
2. Define the relationships
3. Build the graph in NetworkX
4. Traverse for a multi-hop answer
5. Compare against flat RAG

YOU'LL PRODUCE A small invoice knowledge graph (customer → invoice → line item) and a multi-hop answer that flat RAG cannot produce.

Tools: NetworkX, Python, labs/data/invoice_graph.csv · Full step-by-step guide: Learner Guide, Activity 10

© 2026 Tertiary Infotech Academy Pte Ltd

Lunch Break

1 hour

Deploying Models for Inference

1

Managed API endpoints

OpenAI, Gemini, OpenRouter — no infrastructure, per-token pricing, fastest to ship.

2

Serverless functions

Scale to zero and pay per request; watch cold-start latency on the first call.

3

Container platforms

Streamlit Cloud, Railway, Cloud Run — a full app with predictable runtime behaviour.

4

Dedicated GPU

Self-hosted open models; maximum control and privacy, highest fixed cost.

5

The trade-off

Cost, latency, control and data residency — you cannot maximise all four at once.

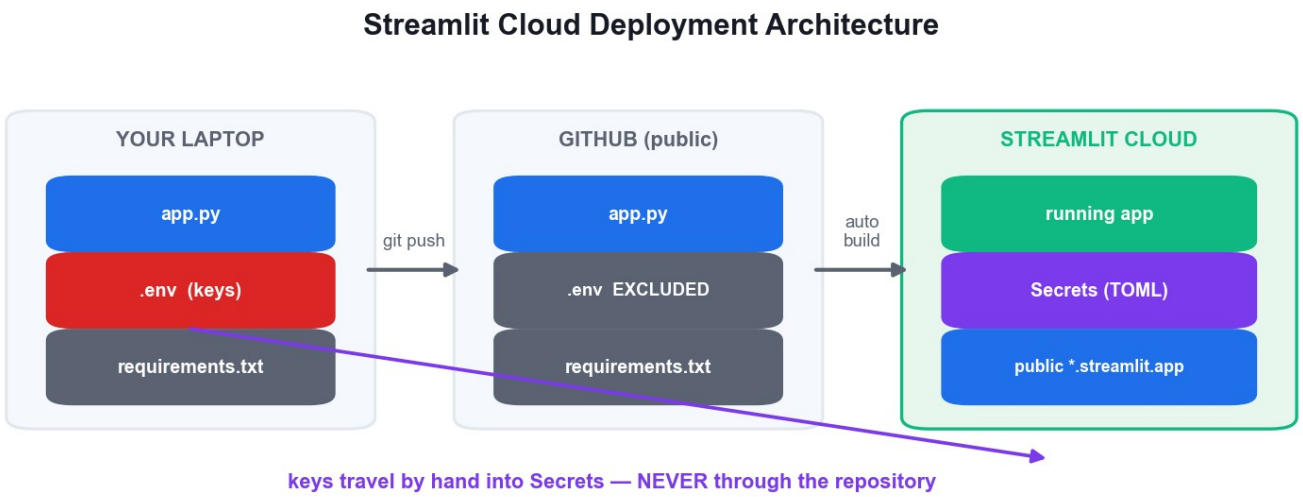
6

For this course

Streamlit Community Cloud plus managed model APIs — production-shaped and free to run.

© 2026 Tertiary Infotech Academy Pte Ltd

Streamlit Cloud Deployment Architecture



Code goes to GitHub

The repository is the deployment unit — Streamlit Cloud builds directly from it.

.env never travels

Your local key file is git-ignored and must never reach the repository.

Secrets are injected

Keys are pasted into the Secrets manager in TOML and mounted at run time.

Public by default

The result is a *.streamlit.app URL anyone can open — so guardrails matter.

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · T2 — PLATFORM COMPARISON

Streamlit Cloud vs Railway

Aspect	Streamlit Community Cloud	Railway
Best for	Streamlit data & AI apps	Any containerised service
Cost	Free for public apps	Usage-based, free trial credit
Deploy from	A public GitHub repo	GitHub, Docker or CLI
Secrets	Secrets manager (TOML)	Environment variables
Scaling	Fixed small instance	Configurable resources
Custom domain	No	Yes

Streamlit Cloud for the fastest path to a shareable AI app; Railway when you need containers, a database or a custom domain.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 11 · 60 MIN

Activity 11 — Swap Model Providers — Gemini and OpenRouter

Resolve real multi-model compatibility issues by pointing the same RAG application at three different providers through the OpenAI-compatible API surface: OpenAI, Google Gemini and OpenRouter. Compare cost, latency and answer quality on identical questions.

1. Run the OpenAI baseline
2. Diff the Gemini variant
3. Set base_url + provider key
4. Run the OpenRouter variant
5. Compare cost / latency / quality

YOU'LL PRODUCE The same RAG app running against three providers, with a comparison of cost, speed and answer quality.

Tools: OpenAI API, Gemini API, OpenRouter, labs/app/invoice_rag_gemini.py, labs/app/invoice_rag_openrouter.py · Full step-by-step guide: Learner Guide, Activity 11

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON ACTIVITY

ACTIVITY 12 · 75 MIN

Activity 12 — Deploy a RAG App to Streamlit Cloud

Take the RAG application from localhost to a public URL on Streamlit Community Cloud: pin the dependencies, push to GitHub, connect the repository, and inject API keys through the Secrets manager rather than committing them — the deployment pattern behind the course demo app.

1. Pin requirements.txt
2. Push to a public GitHub repo
3. Connect Streamlit Cloud to the repo
4. Add keys in Secrets (TOML)
5. Deploy and open the public URL

YOU'LL PRODUCE A publicly reachable Streamlit Cloud URL running YOUR RAG app, with secrets configured and no keys in the repository.

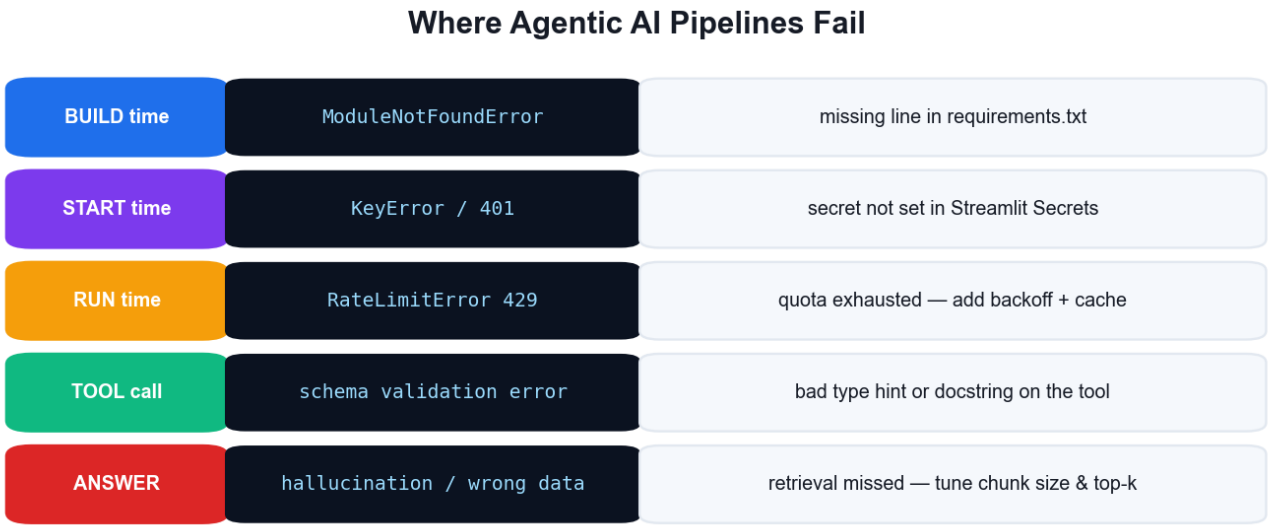
Tools: Streamlit Community Cloud, GitHub, requirements.txt, Streamlit Secrets · Full step-by-step guide: Learner Guide, Activity 12

© 2026 Tertiary Infotech Academy Pte Ltd

Tea Break

15 minutes

Where Agentic AI Pipelines Fail



Read the FIRST error, not the last — everything after it is usually a consequence.

Build vs run

A missing dependency fails at build; a missing secret fails at start — different logs, different fixes.

Read the FIRST error

Everything after the first traceback line is usually a downstream consequence.

Tool errors are schema errors

Missing type hints and vague docstrings break tool calling more often than model quality.

Bad answers are retrieval bugs

If the context was wrong, no amount of prompt tuning will fix the answer.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 13 · 45 MIN

Activity 13 — Diagnose Deployment Failures from Error Logs

Deployments fail in a small number of predictable ways. Deliberately trigger the four most common failures, read the actual log output each produces, and build a reusable symptom → root-cause → fix table for diagnosing Agentic AI pipelines.

1. Break the dependency pin
2. Remove the API secret
3. Force a rate-limit error
4. Break a tool signature
5. Complete the triage table

YOU'LL PRODUCE A completed error-log triage table mapping four real log signatures to their root causes and fixes.

Tools: Streamlit Cloud logs, Python tracebacks, labs/data/error_triage.csv · Full step-by-step guide: Learner Guide, Activity 13

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 14 · 60 MIN

Activity 14 — Deploy the Multi-Agent App and Monitor It

Deploy the multi-agent trip planner from Activity 6 to Streamlit Cloud, add caching and spend controls so a public multi-agent app cannot run away with your API budget, and monitor its behaviour under real use.

1. Deploy trip_planner.py
2. Add st.cache_data to searches
3. Cap requests per session
4. Set a provider spend limit
5. Monitor logs and token usage

YOU'LL PRODUCE A publicly deployed multi-agent app with caching, a request cap and monitored token spend.

Tools: Streamlit Community Cloud, OpenAI usage dashboard, st.cache_data, labs/app/trip_planner.py · Full step-by-step guide: Learner Guide, Activity 14

© 2026 Tertiary Infotech Academy Pte Ltd

LU2 · LEARNING OUTCOME 2 ACHIEVED

Recap — Learning Unit 2

- You can explain and build the full RAG pipeline: chunk, embed, store, retrieve, augment, generate.
- You can tune chunk size and top-k using evidence rather than guesswork.
- You can explain when a knowledge graph beats flat vector retrieval, and why.
- You have deployed a RAG app and a multi-agent app to a public Streamlit Cloud URL.
- You can diagnose build-time, start-time, run-time and tool-call failures from their logs.

© 2026 Tertiary Infotech Academy Pte Ltd

03

LEARNING UNIT 03

Model Alignment and Guardrails

Safeguarding LLMs with guardrails · model alignment for safety · deploying guardrails on Agentic AI applications

© 2026 Tertiary Infotech Academy Pte Ltd

WHY GUARDRAILS

A public agent will be attacked.

The moment your app has a URL, someone will try prompt injection. Guardrails are not optional polish — they are the deployment requirement.

© 2026 Tertiary Infotech Academy Pte Ltd

What Guardrails Defend Against

1. **Prompt injection**
   Instructions hidden in user input or retrieved documents that hijack the agent.

2. **Jailbreaks**
   Role-play and framing attacks that talk the model out of its own safety training.

3. **PII leakage**
   Personal data from your RAG corpus surfacing in an answer it should never appear in.

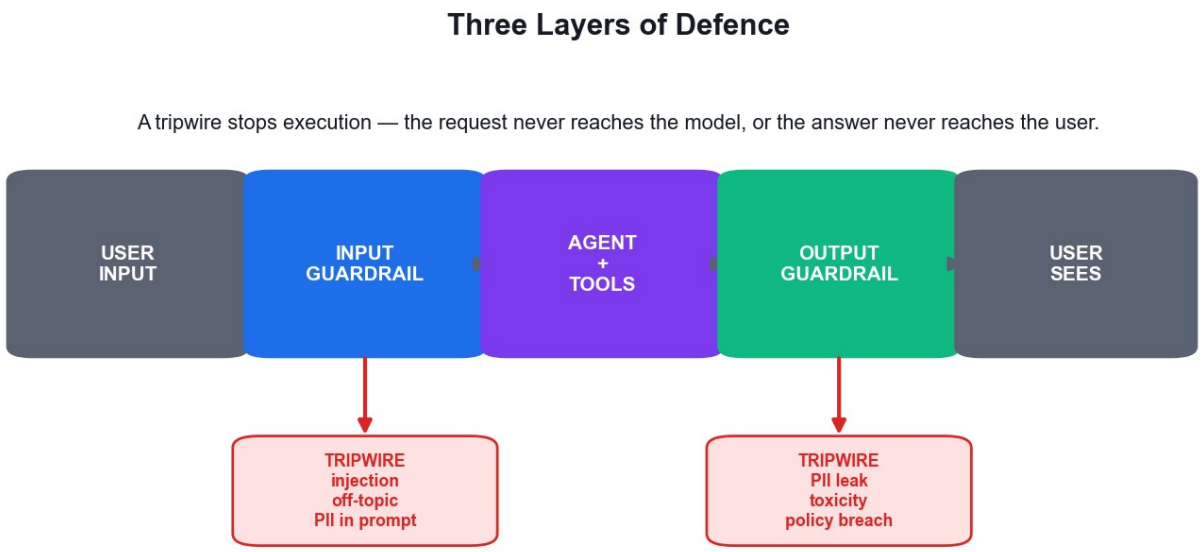
4. **Toxic or unsafe output**
   Content that is harmful, defamatory or breaches your organisation's policy.

5. **Off-topic abuse**
   Your paid API budget spent on requests your application was never built for.

6. **Unsafe tool invocation**
   An agent persuaded to call a real, consequential tool with attacker-chosen arguments.

© 2026 Tertiary Infotech Academy Pte Ltd

Three Layers of Defence



Input guardrails

Validate and classify BEFORE the expensive model call — cheapest place to say no.

The agent itself

Instructions, limited tools and least privilege reduce what can go wrong at all.

Output guardrails

Screen the generated answer for PII, toxicity and policy breaches before display.

Tripwires fail closed

A triggered tripwire raises and stops execution — it does not warn and continue.

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · T1 — TWO APPROACHES

Rule-Based vs AI-Assisted Guardrails

Deterministic

- **Rule-based (regex, allow-lists)**
  - Fast, free and fully deterministic
  - Perfect for formats: emails, NRIC, card numbers
  - Cannot judge intent or context
  - Trivially evaded by rephrasing

Contextual

- **AI-assisted (classifier agent)**
  - Understands intent, paraphrase and context
  - Catches novel attacks a rule never anticipated
  - Costs a model call and adds latency
  - Can itself be prompt-injected — use a small, narrow model

Production systems layer both: regex for the deterministic patterns, a cheap classifier agent for intent.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 15 · 45 MIN

Activity 15 — Attack Your Own Agent — Red-Team the Baseline

You cannot defend what you have not attacked. Run a structured red-team pass against your un-guardrailed agent from Activity 3: prompt injection, jailbreaks, off-topic abuse, PII extraction and unsafe tool invocation — and record every success as a requirement for LU3.

1. Run the un-guardrailed agent
2. Try direct prompt injection
3. Try a role-play jailbreak
4. Try off-topic + PII extraction
5. Log every success as a requirement

YOU'LL PRODUCE A completed red-team log of attacks attempted, which succeeded, and the guardrail each one demands.

Tools: labs/app/simple_chat.py, labs/data/redteam_log.csv · Full step-by-step guide: Learner Guide, Activity 15

© 2026 Tertiary Infotech Academy Pte Ltd

Tea Break

15 minutes

TOPIC 03 · HANDS-ON ACTIVITY

ACTIVITY 16 · 60 MIN

Activity 16 — Implement Input Guardrails with Tripwires

Block malicious input before it reaches the expensive model call. Implement an OpenAI Agents SDK input guardrail that uses a fast, cheap classifier agent to detect off-topic and injection attempts, and raises a tripwire that stops execution entirely.

1. Define the Pydantic output type
2. Build the classifier agent
3. Decorate with @input_guardrail
4. Attach to the main agent
5. Re-run your Activity 15 attacks

YOU'LL PRODUCE An input guardrail that blocks the injection and off-topic attacks you logged in Activity 15.

Tools: OpenAI Agents SDK guardrails, Pydantic, labs/app/guarded_chat.py · Full step-by-step guide: Learner Guide, Activity 16

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON ACTIVITY

ACTIVITY 17 · 60 MIN

Activity 17 — Implement Output Guardrails — PII and Content Filtering

Screen what the agent produces before the user ever sees it. Add an output guardrail that detects and redacts personally identifiable information leaking from the invoice RAG corpus, and blocks unsafe or policy-violating content.

1. Identify PII in the corpus
2. Write regex redaction
3. Add an LLM policy checker
4. Wire the @output_guardrail
5. Verify PII never reaches the UI

YOU'LL PRODUCE An output guardrail that redacts PII from RAG answers and blocks policy-violating responses.

Tools: OpenAI Agents SDK output guardrails, Python re, labs/app/invoice_rag.py · Full step-by-step guide: Learner Guide, Activity 17

© 2026 Tertiary Infotech Academy Pte Ltd

Lunch Break

1 hour

Model Alignment for Safety

1

The HHH goal

Helpful, Harmless and Honest — the three properties alignment training optimises for.

2

RLHF

Reinforcement Learning from Human Feedback: humans rank outputs, a reward model trains the policy.

3

DPO

Direct Preference Optimization learns from preference pairs directly — simpler and cheaper than RLHF.

4

Constitutional AI

The model critiques and revises its own output against an explicit written set of principles.

5

Shallow vs deep alignment

Shallow alignment refuses the obvious phrasing; deep alignment resists rephrased attacks.

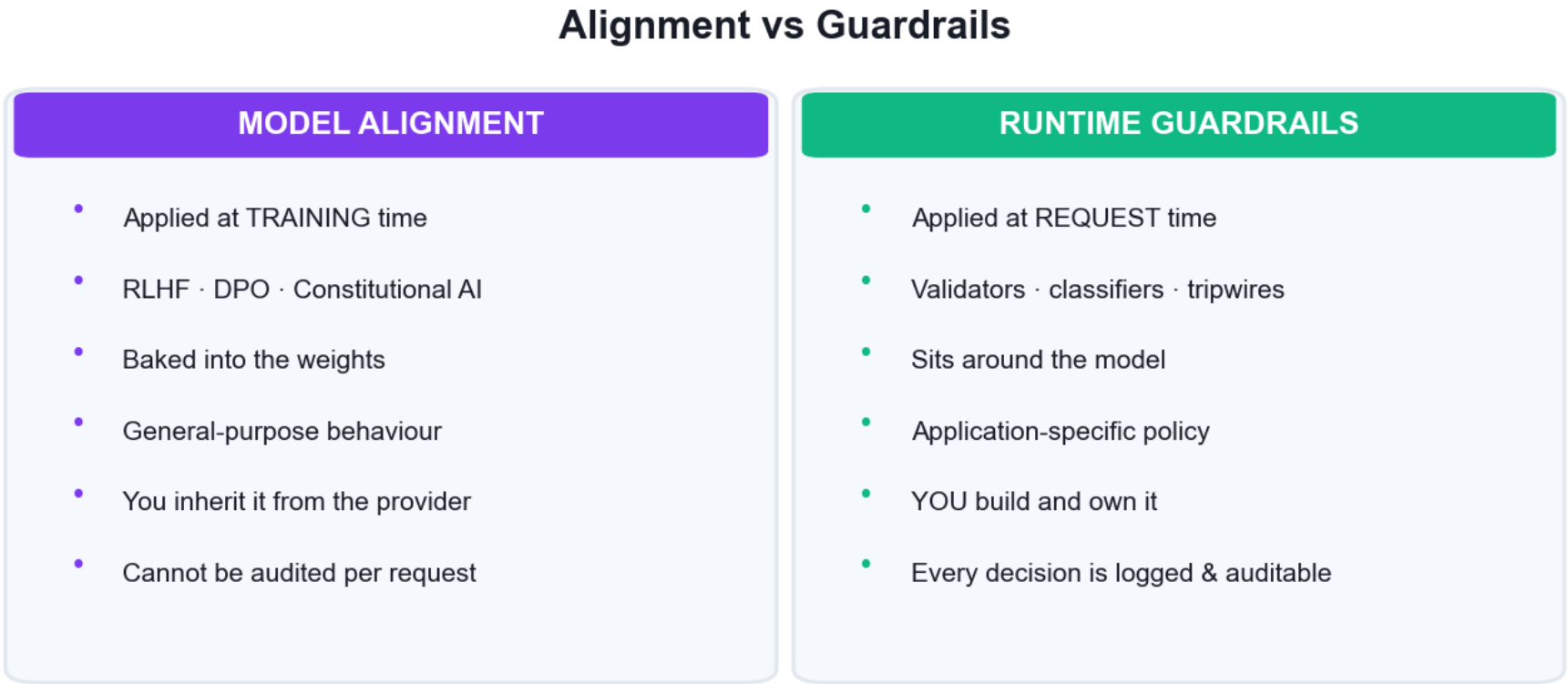
6

Alignment is not enough

Every aligned model can still be jailbroken — which is exactly why you add guardrails.

© 2026 Tertiary Infotech Academy Pte Ltd

Alignment vs Guardrails



Alignment is trained by the model provider and applies generally. Guardrails are built by YOU, enforce your application's policy, and are auditable per request.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 18 · 45 MIN

Activity 18 — Evaluate Model Alignment — Helpful, Harmless, Honest

Alignment is trained into the model; guardrails are added by you. Build a small evaluation suite that scores your agent on helpfulness, harmlessness and honesty, and measure how RLHF-style alignment and your runtime guardrails contribute differently to the result.

1. Write 10 evaluation prompts
2. Score helpfulness 1-5
3. Score harmlessness 1-5
4. Score honesty 1-5
5. Compare guarded vs un-guarded

YOU'LL PRODUCE A scored alignment evaluation across ten prompts, with the helpful/harmless/honest profile of your agent.

Tools: Python, labs/data/alignment_eval.csv · Full step-by-step guide: Learner Guide, Activity 18

© 2026 Tertiary Infotech Academy Pte Ltd

Deploying Guardrails in Production

1

Risk-based routing

Low-risk actions run automatically; high-risk actions require a human approval click.

2

Fail closed

When a guardrail errors or is uncertain, block — never let uncertainty default to allow.

3

Audit everything

Append-only logs of prompt, decision, risk and approver make incidents reconstructable.

4

Monitor continuously

Track block rates and false positives — over-blocking destroys the product's usefulness.

5

Update as attacks evolve

Red-team on a schedule; yesterday's guardrail does not stop tomorrow's phrasing.

6

Guard latency and cost

Every guardrail is an extra call — use small, fast models on the hot path.

© 2026 Tertiary Infotech Academy Pte Ltd

Production Go-Live Checklist

1. Secrets in the Secrets manager — never in the repository
2. Input guardrails block injection and off-topic requests
3. Output filtering redacts PII and unsafe content
4. Human approval gate on every high-risk action
5. Audit log, spend cap and error handling in place

Activity 20 is exactly this checklist, verified by re-running your Activity 15 attacks against the live public URL.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON ACTIVITY

ACTIVITY 19 · 45 MIN

Activity 19 — Add Human Oversight and an Audit Log

Automated guardrails cannot judge every case. Add a risk-based approval gate that routes high-impact or low-confidence actions to a human before execution, and write an audit log that records every guardrail decision for later review.

1. Classify actions by risk
2. Auto-approve low risk
3. Gate high risk on a human click
4. Log every decision to JSONL
5. Review the audit trail

YOU'LL PRODUCE An agent with a human-approval gate on high-risk actions and a persistent JSONL audit log of every decision.

Tools: Streamlit, Python logging, labs/app/guarded_chat.py (the audit log is created when you run this step) · Full step-by-step guide: Learner Guide, Activity 19

© 2026 Tertiary Infotech Academy Pte Ltd

Tea Break

15 minutes

ACTIVITY 20 · 75 MIN

Activity 20 — Deploy the Guardrailed Agentic AI App to Production

Bring the whole course together: deploy the fully guardrailed multi-agent application to Streamlit Cloud with input guardrails, output filtering, human oversight, audit logging and spend controls, then verify the deployed app resists the attacks from Activity 15.

1. Assemble all guardrails in one app
2. Push and deploy to Streamlit Cloud
3. Configure Secrets + spend caps
4. Re-run the Activity 15 attacks
5. Complete the go-live checklist

YOU'LL PRODUCE A publicly deployed, fully guardrailed Agentic AI application that passes a live red-team re-test.

Tools: Streamlit Community Cloud, GitHub, Streamlit Secrets, all prior activity code · Full step-by-step guide: Learner Guide, Activity 20

© 2026 Tertiary Infotech Academy Pte Ltd

LU3 · LEARNING OUTCOME 3 ACHIEVED

Recap — Learning Unit 3

- You can name the concrete threats guardrails defend against, and red-team your own agent for them.
- You have implemented input guardrails with tripwires that block injection and off-topic abuse.
- You have implemented output guardrails that redact PII and block policy-violating content.
- You can distinguish model alignment (training-time) from runtime guardrails (yours, auditable).
- You have deployed a fully guardrailed agentic app that survives a live red-team re-test.

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP-UP

Course Summary & Next Steps

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNING OUTCOMES DELIVERED

What You Achieved

1. **Agentic AI frameworks**
   Built agents in CrewAI, AutoGen and the OpenAI Agents SDK, and justified framework choice. (LO1)
2. **Multi-agent workflows**
   Designed and implemented sequential, router, parallel and reflection orchestration. (LO1)
3. **RAG and context augmentation**
   Built a full RAG pipeline, tuned retrieval and modelled a knowledge graph. (LO2)
4. **Cloud deployment**
   Deployed RAG and multi-agent apps to public Streamlit Cloud URLs with managed secrets. (LO2)
5. **Error diagnosis**
   Triaged build, start, run-time and tool-call failures from real logs. (LO2)
6. **Guardrails and alignment**
   Shipped input, output and human-oversight controls that survive a red-team re-test. (LO3)

© 2026 Tertiary Infotech Academy Pte Ltd

NEXT STEPS

Keep Building After Today

1

Re-deploy from scratch

Rebuild Activity 20 from an empty folder — that is when it truly becomes yours.

2

Bring your own data

Swap the invoice PDFs for your organisation's documents and re-tune retrieval.

3

Add a framework

Port your trip planner to CrewAI or AutoGen and compare what changes.

4

Red-team monthly

Attacks evolve; re-run and extend your red-team log on a schedule.

5

Track the SDKs

These libraries move fast — follow the release notes for the ones you deploy.

6

Watch your spend

Every public agent needs a cache, a session cap and a hard provider spend limit.

© 2026 Tertiary Infotech Academy Pte Ltd

CONTINUE THE AI JOURNEY

Recommended Follow-On Courses

1

Build RAG and AI Agents with LangChain and LangGraph

WSQ funded · Tertiary Infotech Academy

2

Generative AI Application Development with Python

WSQ funded · Tertiary Infotech Academy

3

Prompt Engineering for Generative AI

WSQ funded · Tertiary Infotech Academy

© 2026 Tertiary Infotech Academy Pte Ltd

WE'RE HERE TO HELP — DURING AND AFTER THE CLASS

Support

1

Email

enquiry@tertiaryinfotech.com

2

Telephone

+65 6100 0613

3

Website

www.tertiarycourses.com.sg

4

Course portal

https://lms-tms.tertiaryinfotech.com

© 2026 Tertiary Infotech Academy Pte Ltd

FINAL ASSESSMENT

Assessment

WA — Written

- Short-Answer Questions (SAQ)
- 1 hour · open book
- Knowledge across LU1, LU2 and LU3

PP — Practical

- Practical Performance tasks
- 1 hour · open book
- Build / deploy / safeguard, from the activities

Before you start

- Take the Assessment digital attendance (TRAQOM)
- Submit your answers on the LMS
- https://lms-tms.tertiaryinfotech.com/

© 2026 Tertiary Infotech Academy Pte Ltd

ON ASSESSMENT DAY

Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Sit WA (SAQ) 1 h, then PP 1 h — open book
4. Submit your answers on the LMS
5. Sign the Assessment Summary Record

© 2026 Tertiary Infotech Academy Pte Ltd

Digital Attendance (Mandatory)

1. Trainer displays the SSG digital attendance QR code
2. Scan it with your mobile phone camera
3. Submit your attendance — AM, PM and Assessment
4. Repeat every session, every day
5. 75% attendance minimum for assessment and funding

Digital attendance is mandatory for WSQ-funded courses — missing a scan puts your funding at risk.

© 2026 Tertiary Infotech Academy Pte Ltd

HAPPY BUILDING

Thank You!

You can now design, build, deploy and safeguard Agentic AI applications — go and ship one.

© 2026 Tertiary Infotech Academy Pte Ltd